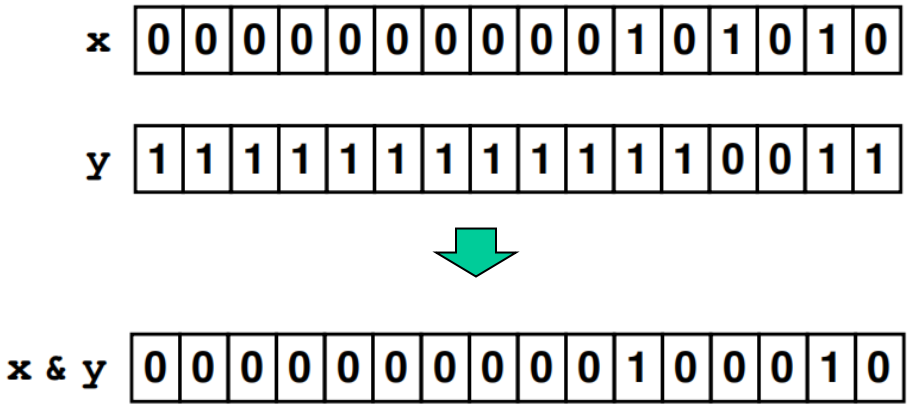
Bitwise Operators

- If you know your client is working with sets of characters, you can implement the set operators extremely efficiently by storing the set as an array of bits and then manipulating the bits all at once using C++'s ***bitwise operators***.
- The bitwise operators are summarized in the following table and then described in more detail on the next few slides:

Inter-section	*x* **&** *y*	Bitwise AND. The result has a 1 bit wherever both *x* and *y* have 1s.
Union	*x* **|** *y*	Bitwise OR. The result has a 1 bit wherever either *x* or *y* have 1s.
	x **^** *y*	Exclusive OR. The result has a 1 bit wherever *x* and *y* differ.
	~*x*	Bitwise NOT. The result has a 1 bit wherever *x* has a 0.
	x **<<** *n*	Left shift. Shift the bits in *x* left *n* positions, shifting in 0s.
	x **>>** *n*	Right shift. Shift *x* right *n* bits

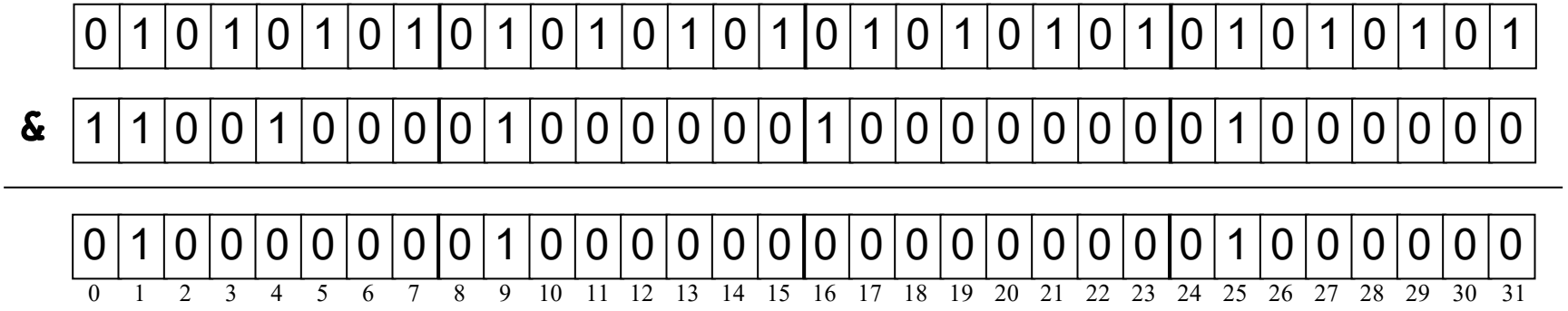
The Bitwise AND Operators



x & *y*	Bitwise AND. The result has a 1 bit wherever both *x* and *y* have 1s.
x | *y*	Bitwise OR. The result has a 1 bit wherever either *x* or *y* have 1s.
x ^ *y*	Exclusive OR. The result has a 1 bit wherever *x* and *y* differ.
~*x*	Bitwise NOT. The result has a 1 bit wherever *x* has a 0.
x << *n*	Left shift. Shift the bits in *x* left *n* positions, shifting in 0s.
x >> *n*	Right shift. Shift *x* right *n* bits (logical shift if *x* is unsigned).

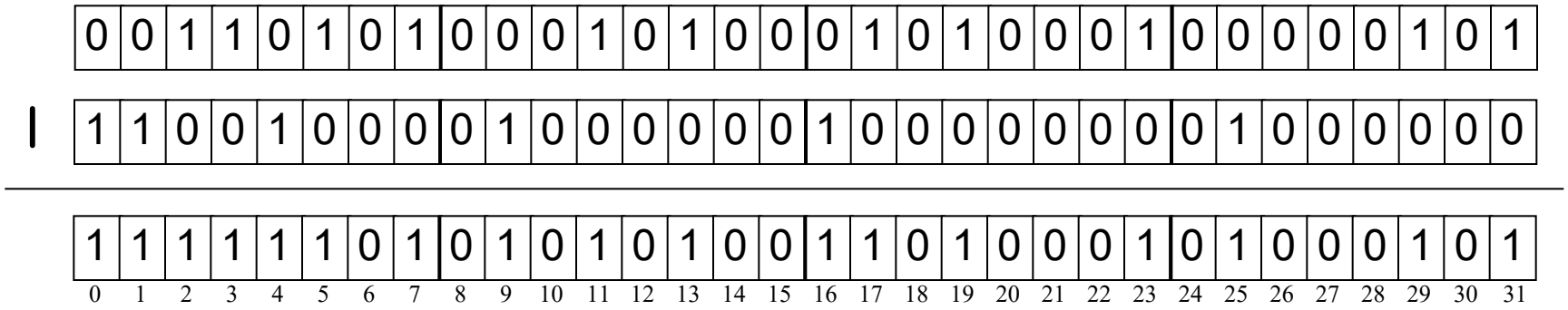
The Bitwise AND Operator

- The bitwise AND operator (`&`) takes two integer operands, *x* and *y*, and computes a result that has a 1 bit in every position in which both *x* and *y* have 1 bits.
- The primary application of the `&` operator is to ***select*** certain bits in an integer, clearing the unwanted bits to 0. This operation is called ***masking***.
- In the context of sets, the `&` operator performs an intersection operation as $A \cap B$

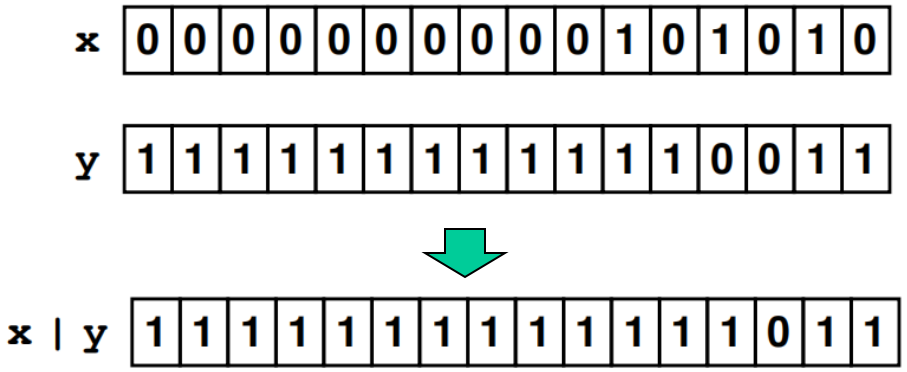


The Bitwise OR Operator

- The bitwise OR operator (`|`) takes two integer operands, *x* and *y*, and computes a result that has a 1 bit in every position which either *x* or *y* has a 1 bit (or if both do, i.e., non-exclusive).
- The primary use of the `|` operator is to ***assemble*** a single integer value from other values, each of which contains a subset of the desired bits.
- In the context of sets, the `|` operator performs a union, as in the following calculation of **A ∪ B**:



The Bitwise OR

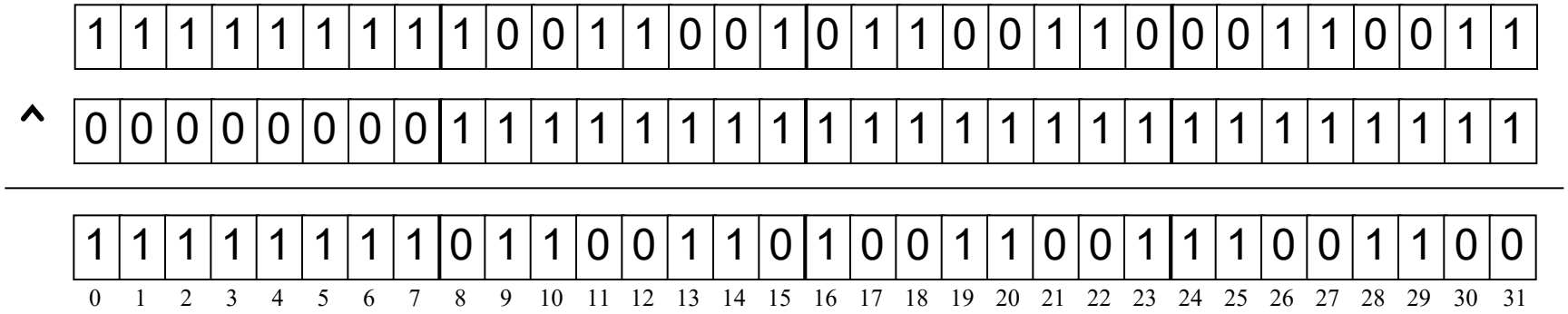


x & *y*	Bitwise AND. The result has a 1 bit wherever both *x* and *y* have 1s.
x | *y*	Bitwise OR. The result has a 1 bit wherever either *x* or *y* have 1s.
x ^ *y*	Exclusive OR. The result has a 1 bit wherever *x* and *y* differ.
~*x*	Bitwise NOT. The result has a 1 bit wherever *x* has a 0.
x << *n*	Left shift. Shift the bits in *x* left *n* positions, shifting in 0s.
x >> *n*	Right shift. Shift *x* right *n* bits (logical shift if *x* is unsigned).

The Exclusive OR Operator

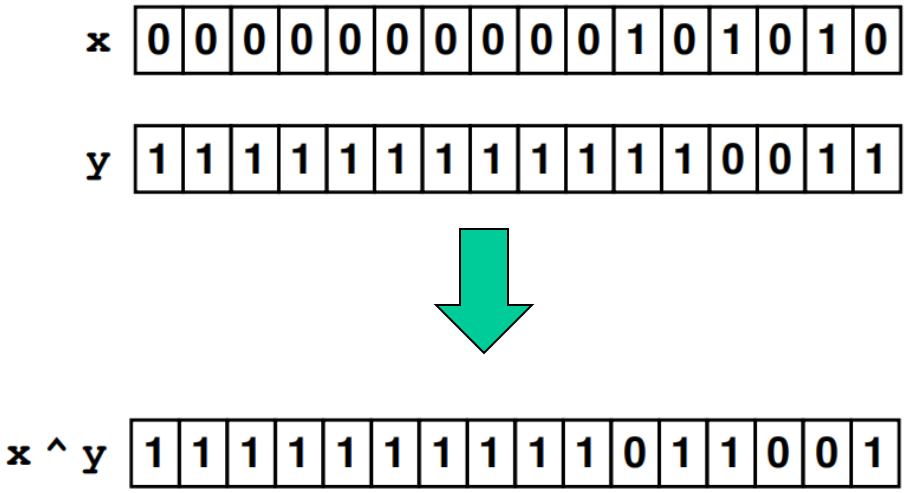
or but not both

- The exclusive OR or XOR operator (^) takes two integer operands, *x* and *y*, and computes a result that has a 1 bit in every position in which *x* and *y* have different bit values.
- The XOR operator has many applications in programming, most of which are beyond the scope of this text.
- The following example ***flips*** all the bits in the rightmost three bytes of a word:



- Question: What if we apply the same XOR mask twice?

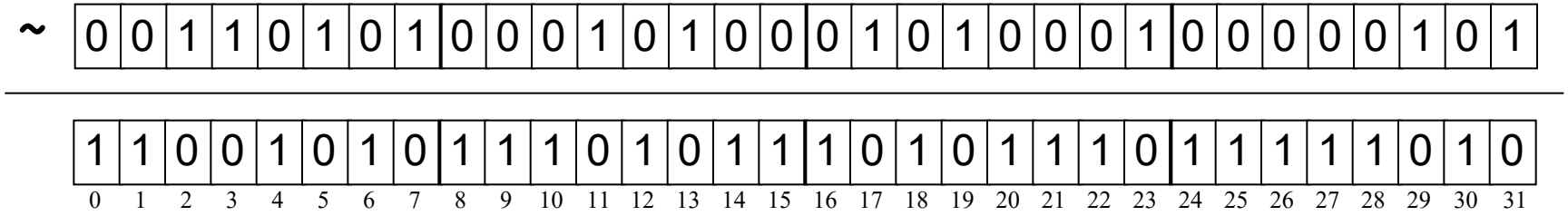
The Bitwise XOR Operators



x & *y*	Bitwise AND. The result has a 1 bit wherever both *x* and *y* have 1s.
x | *y*	Bitwise OR. The result has a 1 bit wherever either *x* or *y* have 1s.
x ^ *y*	Exclusive OR. The result has a 1 bit wherever *x* and *y* differ.
~*x*	Bitwise NOT. The result has a 1 bit wherever *x* has a 0.
x << *n*	Left shift. Shift the bits in *x* left *n* positions, shifting in 0s.
x >> *n*	Right shift. Shift *x* right *n* bits (logical shift if *x* is unsigned).

The Bitwise NOT Operator

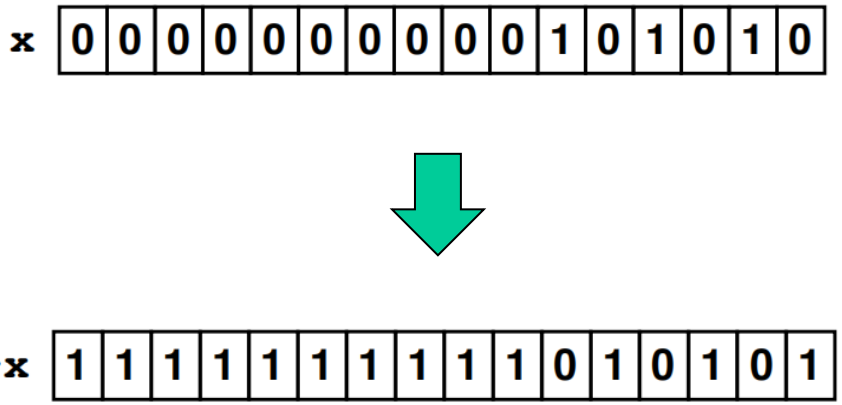
- The bitwise NOT operator (**~**) takes a single operand *x* and returns a value that has a 1 wherever *x* has a 0, and vice versa.
- You can use the bitwise NOT operator to create a mask in which you mark the bits you want to eliminate as opposed to the ones you want to preserve.
- The **~** operator creates the ***complement*** of a set, as shown with the following diagram of **~A**:



x ^ y : Flip the bits in x, when y has 1

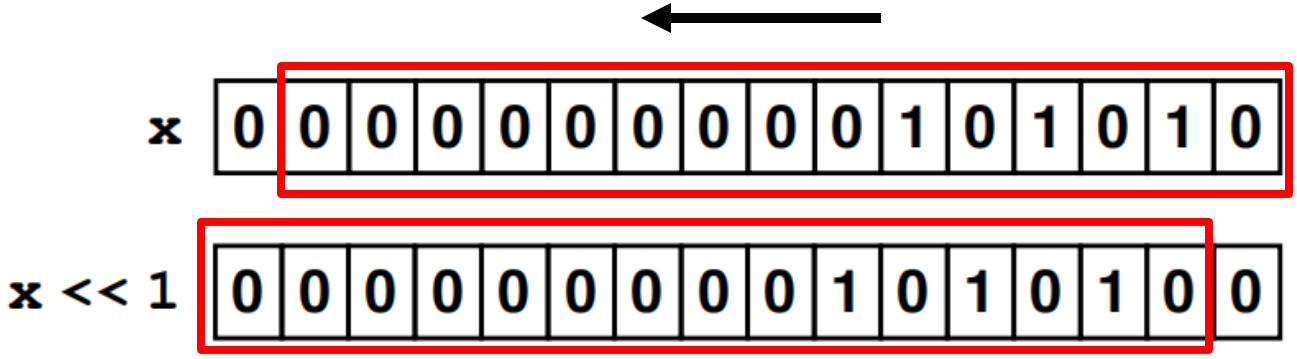
~x : Flip all bits in x

The Bitwise NOT Operators



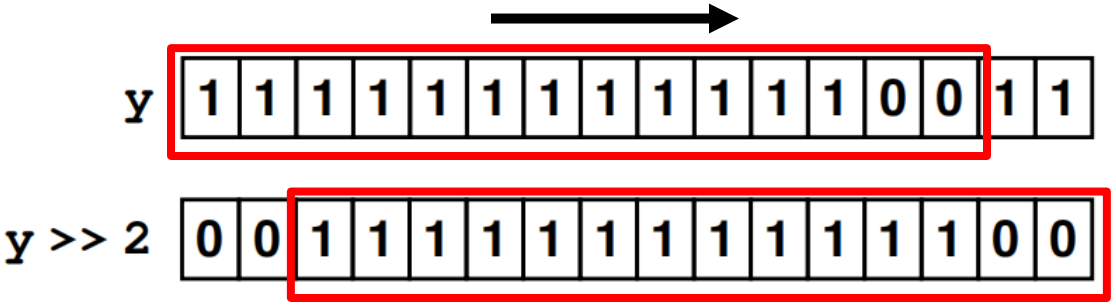
x & *y*	Bitwise AND. The result has a 1 bit wherever both *x* and *y* have 1s.
x | *y*	Bitwise OR. The result has a 1 bit wherever either *x* or *y* have 1s.
x ^ *y*	Exclusive OR. The result has a 1 bit wherever *x* and *y* differ.
~*x*	Bitwise NOT. The result has a 1 bit wherever *x* has a 0.
x << *n*	Left shift. Shift the bits in *x* left *n* positions, shifting in 0s.
x >> *n*	Right shift. Shift *x* right *n* bits (logical shift if *x* is unsigned).

Bitwise Shift Operators



x & y	Bitwise AND. The result has a 1 bit wherever both x and y have 1s.
x | y	Bitwise OR. The result has a 1 bit wherever either x or y have 1s.
x ^ y	Exclusive OR. The result has a 1 bit wherever x and y differ.
~x	Bitwise NOT. The result has a 1 bit wherever x has a 0.
x << n	Left shift. Shift the bits in x left n positions, shifting in 0s.
x >> n	Right shift. Shift x right n bits (logical shift if x is unsigned).

Bitwise Shift Operators



x & *y*	Bitwise AND. The result has a 1 bit wherever both *x* and *y* have 1s.
x | *y*	Bitwise OR. The result has a 1 bit wherever either *x* or *y* have 1s.
x ^ *y*	Exclusive OR. The result has a 1 bit wherever *x* and *y* differ.
~*x*	Bitwise NOT. The result has a 1 bit wherever *x* has a 0.
x << *n*	Left shift. Shift the bits in *x* left *n* positions, shifting in 0s.
x >> *n*	Right shift. Shift *x* right *n* bits (logical shift if *x* is unsigned).

Bitwise Shift vs Set Operations

- The ***union*** of two sets A and B ($A \cup B$) consists of all elements in either A or B or both. （A|B）
- The ***intersection*** of A and B ($A \cap B$) consists of all elements in both A and B. （A&B）
- The ***set difference*** of A and B ($A - B$) consists of all elements in A but not in B. （A&~B）

x `&` y	Bitwise AND. The result has a 1 bit wherever both x and y have 1s.
x `|` y	Bitwise OR. The result has a 1 bit wherever either x or y have 1s.
x `^` y	Exclusive OR. The result has a 1 bit wherever x and y differ.
`~`x	Bitwise NOT. The result has a 1 bit wherever x has a 0.
x `<<` n	Left shift. Shift the bits in x left n positions, shifting in 0s.
x `>>` n	Right shift. Shift x right n bits (logical shift if x is unsigned).